

DE .ONNX A .HEF EN HAILO-8

Guía completa para Windows

Raspberry Pi 5 + Hailo-8 AI Hat | YOLOv8n-Pose | 12 keypoints

Centro	IES Politécnico Jesús Marín, Málaga
Proyecto	Detección orientación giroscopio drones (Roll/Pitch/Yaw)
Hardware objetivo	Raspberry Pi 5 + Hailo-8 AI Hat
Imagen Docker	hailo8_ai_sw_suite_2025-10:1
Hailo DFC / HailoRT	v3.33.0 / v4.23.0
Modelo	giroscopio_pose_best_n_g1.onnx (YOLOv8n-Pose, 640×640, 12 KP)
Workspace Windows	C:\Users\TuUsuario\hailo_workspace
Workspace en Docker	/workspace (montado con -v)

Índice

PASO 0 — Descargar el modelo desde Google Colab

PARTE 1 — Preparación del entorno Windows

- 1.1 Instalar WSL2
- 1.2 Instalar Docker Desktop
- 1.3 Verificar Docker

PARTE 2 — Cargar la imagen Hailo en Docker

- 2.1 Descargar imagen desde el portal Hailo
- 2.2 Cargar la imagen con docker load

PARTE 3 — Preparar el workspace

- 3.1 Estructura de carpetas
- 3.2 Copiar archivos al workspace

PARTE 4 — Pipeline dentro del contenedor (idéntico a Ubuntu)

- 4.1 Arrancar el contenedor (PowerShell)
- 4.2 Convertir imágenes de calibración a .npy
- 4.3 Parsear: .onnx → .har
- 4.4 Crear el script de optimización (.alls)
- 4.5 Optimizar y cuantizar: .har → .har optimizado
- 4.6 Compilar: .har optimizado → .hef
- 4.7 Guardar archivos y salir del contenedor

PARTE 5 — Copiar el .hef a la Raspberry Pi 5

PARTE 6 — Referencia rápida

- 6.1 Chuleta: secuencia completa en PowerShell
- 6.2 Tabla de errores y soluciones
- 6.3 Archivos generados y para qué sirve cada uno

PASO 0 — Descargar el modelo desde Google Colab

Antes de empezar el pipeline en Windows necesitas el archivo .onnx que generaste durante el entrenamiento en Google Colab. Sigue uno de estos métodos según cómo hayas guardado el modelo:

Método A — Descarga directa desde el panel de archivos de Colab

Este método funciona si el archivo .onnx está en el disco efímero de Colab (no en Drive). Es el más rápido pero el archivo desaparece cuando se cierra la sesión.

En la celda de entrenamiento de Colab, exporta el modelo a .onnx:

```
# En Colab - exportar el modelo entrenado a ONNX
model.export(format='onnx', imgsiz=640, simplify=True)
# El archivo se genera en: runs/pose/train/weights/best.onnx
# Cópialo con un nombre descriptivo:
import shutil
shutil.copy('runs/pose/train/weights/best.onnx',
            'giroscopio_pose_best_n_g1.onnx')
print('Listo: giroscopio_pose_best_n_g1.onnx')
```

Luego, en el panel lateral izquierdo de Colab:

- Haz clic en el icono de carpeta 📁
- Localiza giroscopio_pose_best_n_g1.onnx
- Clic derecho → Descargar
- Guárdalo en C:\Users\TuUsuario\hailo_workspace\

Método B — Guardar en Google Drive (recomendado si usas Colab Pro)

Este método es más seguro porque el archivo persiste aunque se cierre Colab.

```
# En Colab - montar Drive y copiar el modelo
from google.colab import drive
drive.mount('/content/drive')

import shutil
shutil.copy('runs/pose/train/weights/best.onnx',
            '/content/drive/MyDrive/giroscopio_pose_best_n_g1.onnx')
print('Guardado en Google Drive')
```

Después descárgalo desde Google Drive a tu Windows:

- Abre drive.google.com en el navegador
- Localiza giroscopio_pose_best_n_g1.onnx
- Clic derecho → Descargar
- Guárdalo en C:\Users\TuUsuario\hailo_workspace\

i Nombre del archivo

El nombre exacto del modelo es: giroscopio_pose_best_n_g1.onnx
Todos los scripts y comandos de esta guía usan ese nombre.
Si el tuyo tiene un nombre diferente, cámbialo o ajusta los comandos.

PARTE 1 — Preparación del entorno Windows

1.1 Instalar WSL2

WSL2 (Windows Subsystem for Linux 2) es necesario para que Docker Desktop funcione correctamente con imágenes Linux como la de Hailo. Abre PowerShell como Administrador y ejecuta:

```
# Abrir PowerShell como Administrador y ejecutar:
wsl --install

# Si WSL ya estaba instalado, actualiza a la versión 2:
wsl --set-default-version 2
```

Reinicia el ordenador cuando se solicite. Tras el reinicio, se abrirá automáticamente una ventana para crear el usuario de Ubuntu en WSL — puedes poner cualquier nombre de usuario y contraseña.

! ¿Por qué WSL2 y no Hyper-V?

La imagen Docker de Hailo es un contenedor Linux. WSL2 ofrece mejor rendimiento para este tipo de contenedores en Windows que el backend Hyper-V clásico. Además, WSL2 es la opción por defecto en Docker Desktop desde 2021.

1.2 Instalar Docker Desktop

Docker Desktop es la aplicación oficial para usar Docker en Windows. Incluye una interfaz gráfica y el motor Docker completo.

- Descarga Docker Desktop desde: <https://www.docker.com/products/docker-desktop/>
- Ejecuta el instalador (Docker Desktop Installer.exe)
- En el instalador, asegúrate de que 'Use WSL 2 instead of Hyper-V' está MARCADO
- Finaliza la instalación y reinicia si se solicita
- Abre Docker Desktop desde el menú inicio o el escritorio
- Espera a que arranque el motor — verás el icono de la ballena 🐳 en la barra de tareas

⚠ Requisitos de hardware

Docker Desktop con WSL2 requiere Windows 10 versión 2004 o superior (Build 19041+), o Windows 11. Necesitas al menos 8 GB de RAM y 20 GB de espacio libre en disco. La imagen de Hailo pesa ~17 GB, así que asegúrate de tener suficiente espacio.

1.3 Verificar Docker

Abre una ventana de PowerShell (no hace falta que sea Administrador) y verifica que Docker funciona:

```
# Verificar versión de Docker
docker --version

# Salida esperada: Docker version 26.x.x, build ...
```

```
# Verificar que el motor está corriendo
docker info
# Si ves 'Server: Docker Desktop' el motor está activo
```

 Docker Desktop debe estar abierto

A diferencia de Ubuntu donde Docker corre como servicio del sistema, en Windows Docker Desktop debe estar en ejecución para poder usar Docker. Si ves el error 'Cannot connect to the Docker daemon', abre Docker Desktop y espera a que el motor arranque (el icono de la ballena se vuelve estático).

PARTE 2 — Cargar la imagen Hailo en Docker

2.1 Descargar imagen desde el portal Hailo

La imagen Docker con el compilador Hailo (Dataflow Compiler) no está en Docker Hub. Hay que descargarla del portal oficial de desarrolladores de Hailo.

- Ve a: <https://hailo.ai/developer-zone/>
- Inicia sesión (requiere registro gratuito)
- Busca la sección de descargas de Software Suite
- Descarga la versión que corresponda al firmware de tu chip Hailo-8
- El archivo descargado es un .tar.gz (en nuestro caso: `hailo8_ai_sw_suite_2025-10.tar.gz`)

i ¿Qué versión descargar?

Descarga siempre la versión que coincida con el HailoRT instalado en tu Raspberry Pi 5.

En nuestro proyecto usamos: `hailo8_ai_sw_suite_2025-10.tar.gz`

Que corresponde a Hailo DFC v3.33.0 / HailoRT v4.23.0.

Si descargas una versión diferente, el .hef generado puede no ser compatible con la RPi.

2.2 Cargar la imagen con docker load

Una vez descargado el archivo .tar.gz, cárgalo en Docker desde PowerShell. Asegúrate de estar en la carpeta donde está el archivo:

```
# Navegar a la carpeta donde descargaste el .tar.gz
cd C:\Users\TuUsuario\Downloads

# Cargar la imagen en Docker (puede tardar varios minutos, pesa ~17 GB)
docker load -i hailo8_ai_sw_suite_2025-10.tar.gz

# Verificar que se cargó correctamente
docker images
```

Debes ver algo como:

REPOSITORY	TAG	IMAGE ID	SIZE
hailo8_ai_sw_suite_2025-10	1	3a94fa154926	17.5GB

⚠ El tag es :1, no :latest

Docker asume el tag :latest si no especificas ninguno. Como la imagen tiene tag :1, siempre debes incluirlo explícitamente: `hailo8_ai_sw_suite_2025-10:1`

Sin el tag, Docker busca :latest en internet y falla con 'pull access denied'.

Diferencia con Ubuntu: en Ubuntu se usa 'docker load <' (redirección del shell).

En PowerShell se usa 'docker load -i' (flag explícito). El resultado es idéntico.

PARTE 3 — Preparar el workspace

3.1 Estructura de carpetas

Crea la estructura de carpetas necesaria en Windows. Puedes hacerlo desde PowerShell o manualmente con el Explorador de archivos:

```
# En PowerShell: crear las carpetas
New-Item -ItemType Directory -Force -Path C:\Users\TuUsuario\hailo_workspace
New-Item -ItemType Directory -Force -Path C:\Users\TuUsuario\hailo_workspace\calib
```

La estructura debe quedar así:

```
C:\Users\TuUsuario\hailo_workspace\
├─ giroscopio_pose_best_n_g1.onnx  ← modelo entrenado (del Paso 0)
└─ calib\                          ← imágenes de calibración (100-300 JPG/PNG)
    ├─ imagen_001.jpg
    ├─ imagen_002.jpg
    └─ ...
```

3.2 Copiar archivos al workspace

- Copia giroscopio_pose_best_n_g1.onnx (descargado del Paso 0) a hailo_workspace\
- Copia tus imágenes de calibración a hailo_workspace\calib\

i Sobre las imágenes de calibración

Deben ser imágenes representativas del uso real: el giroscopio en distintas orientaciones, iluminaciones y fondos. Con 100-300 imágenes es suficiente. Formato: JPG o PNG. NO es necesario que sean exactamente 640×640; el script de conversión las redimensiona automáticamente.

PARTE 4 — Pipeline dentro del contenedor

Los pasos dentro del contenedor son idénticos en Windows y Ubuntu. El contenedor es un entorno Linux puro independientemente del sistema operativo del host. Las únicas diferencias están en el comando de arranque.

4.1 Arrancar el contenedor (PowerShell)

Abre PowerShell y ejecuta el siguiente comando. Sustituye TuUsuario por tu nombre de usuario de Windows real:

```
# En PowerShell – el carácter de continuación es el acento grave (`)
docker run -it --rm `
-v C:/Users/TuUsuario/hailo_workspace:/workspace `
hailo8_ai_sw_suite_2025-10:1 bash
```

Si prefieres usar CMD (símbolo del sistema), el carácter de continuación es ^:

```
:: En CMD – el carácter de continuación es el circunflejo (^)
docker run -it --rm ^
-v C:/Users/TuUsuario/hailo_workspace:/workspace ^
hailo8_ai_sw_suite_2025-10:1 bash
```

El prompt del contenedor cambia a algo como:

```
(hailo_virtualenv) hailo@d07f0c1431f6:/local/workspace$
```

i Diferencias de sintaxis Windows vs Ubuntu

PowerShell: continuación de línea con acento grave (`)

CMD: continuación de línea con circunflejo (^)

Ubuntu bash: continuación de línea con barra invertida (\)

Rutas de Windows: usa barras / en lugar de \ cuando se pasan a Docker.

Todo lo demás dentro del contenedor es idéntico a Ubuntu.

⚠ REGLA DE ORO: trabaja siempre desde ~

/workspace es de solo lectura para el usuario hailo dentro del contenedor.

Si intentas ejecutar comandos hailo desde /workspace obtendrás:

Permission denied: /workspace/pyhailort.log

Ejecuta SIEMPRE los comandos hailo desde ~ (home del usuario hailo).

Apunta a /workspace solo para LEER archivos de entrada.

Lo primero al entrar al contenedor:

```
cd ~
ls /workspace # debe mostrar tu .onnx y la carpeta calib/
```

4.2 Convertir imágenes de calibración a .npy

El optimizador Hailo no acepta JPG/PNG directamente. Necesita arrays NumPy (.npv). Este paso es obligatorio y solo hay que hacerlo una vez por sesión (los archivos se pierden al salir del contenedor con --rm).

```
python3 << 'EOF'
import os, numpy as np
from PIL import Image
calib_dir = '/workspace/calib/'
out_dir = os.path.expanduser('~/.calib_npy/')
os.makedirs(out_dir, exist_ok=True)
files = [f for f in os.listdir(calib_dir) if f.lower().endswith(('.jpg', '.png'))]
print(f'Convirtiendo {len(files)} imágenes...')
for i, fname in enumerate(files):
    img = Image.open(os.path.join(calib_dir, fname)).convert('RGB')
    img = img.resize((640, 640))
    np.save(os.path.join(out_dir, f'calib_{i:04d}.npv'), np.array(img,
dtype=np.float32))
print(f'Listo: {len(files)} archivos .npv en {out_dir}')
EOF
```

Salida esperada:

```
Convirtiendo 299 imágenes...
Listo: 299 archivos .npv en /home/hailo/calib_npy/
```

⚠ Error StopIteration si se omite este paso

Si pasas --calib-set-path apuntando a una carpeta de JPGs obtendrás:

StopIteration en npv_dir_to_dataset

El error con .npv mal generados es:

Couldn't detect CalibrationDataType

Ambos indican que las imágenes no están en el formato correcto.

4.3 Parsear: .onnx → .har

Este paso traduce el modelo ONNX al formato intermedio de Hailo. Es crítico especificar los 9 nodos finales correctos. Sin ellos el compilador falla con 'No valid partition found'.

```
# Asegúrate de estar en ~
cd ~

hailo parser onnx /workspace/giroscopio_pose_best_n_g1.onnx \
--net-name giroscopio \
--end-node-names \
/model.22/cv2.0/cv2.0.2/Conv \
/model.22/cv3.0/cv3.0.2/Conv \
/model.22/cv4.0/cv4.0.2/Conv \
/model.22/cv2.1/cv2.1.2/Conv \
```

/model.22/cv3.1/cv3.1.2/Conv \
/model.22/cv4.1/cv4.1.2/Conv \
/model.22/cv2.2/cv2.2.2/Conv \
/model.22/cv3.2/cv3.2.2/Conv \
/model.22/cv4.2/cv4.2.2/Conv

Salida esperada:

```
[info] End nodes mapped from original model: '/model.22/cv2.0/cv2.0.2/Conv', ...
[info] Translation completed on ONNX model giroscopio
[info] Saved HAR to: /home/hailo/giroscopio.har
```

Verificar que el archivo se generó:

```
ls -lh ~/giroscopio.har    # debe mostrar ~13 MB
```

i Por qué estos 9 nodos y no otros

El parser automático elige nodos Concat/Mul/Sigmoid como salida, lo que hace que el modelo no quepa en el chip (error: No valid partition found).

Los nodos cv2/cv3/cv4 Conv son los que usa el Hailo Model Zoo para YOLOv8-Pose y permiten que el compilador divida el modelo en dos contextos que sí caben en el Hailo-8.

4.4 Crear el script de optimización (.alls)

El archivo .alls contiene los parámetros de cuantización y optimización. Créalo ejecutando este comando dentro del contenedor:

```
cat > ~/giroscopio.alls << 'EOF'
normalization1 = normalization([0.0, 0.0, 0.0], [255.0, 255.0, 255.0])
change_output_activation(conv71, sigmoid)
change_output_activation(conv58, sigmoid)
change_output_activation(conv44, sigmoid)
pre_quantization_optimization(equalization, policy=disabled)
quantization_param(output_layer3, precision_mode=a16_w16)
quantization_param(output_layer6, precision_mode=a16_w16)
quantization_param(output_layer9, precision_mode=a16_w16)
post_quantization_optimization(finetune, policy=enabled, learning_rate=0.00015)
EOF
```

i Qué hace el script .alls

Normaliza la entrada (divide por 255), fuerza activación sigmoid en las capas de confianza de detección, mantiene 16 bits en las capas de salida de keypoints (a16_w16) para mayor precisión, y activa fine-tuning post-cuantización.

Es el mismo script que usa el Hailo Model Zoo para YOLOv8-Pose.

El archivo giroscopio.alls también se guarda en tu carpeta hailo_workspace en Windows.

4.5 Optimizar y cuantizar: .har → .har optimizado

Este paso cuantiza el modelo de FP32 a INT8 usando las imágenes de calibración y aplica fine-tuning para recuperar precisión. Es el paso más lento (~13 minutos en un i7-9700).

```
hailo optimize ~/giroscopio.har \
--hw-arch hailo8 \
--calib-set-path ~/calib_npy/ \
--model-script ~/giroscopio.alls
```

Verás barras de progreso. Al terminar (~13 min):

```
Calibration: 100%|██████████| 64/64 [00:27<00:00]
[info] Model Optimization Algorithm Quantization-Aware Fine-Tuning is done
(00:13:21)
[info] Model Optimization is done
[info] Saved HAR to: /home/hailo/giroscopio_optimized.har
```

4.6 Compilar: .har optimizado → .hef

El último paso genera el binario final para el chip Hailo-8. Tarda ~8 minutos.

```
hailo compiler ~/giroscopio_optimized.har --hw-arch hailo8
```

Al terminar verás la utilización de cada cluster del chip:

```
[info] Successful Mapping (allocation time: 8m 6s)
[info] Compiling kernels of giroscopio_context_0...
[info] Compiling kernels of giroscopio_context_1...
[info] Successful Compilation (compilation time: 8s)
[info] Saved HEF to: /home/hailo/giroscopio.hef
[info] Saved HAR to: /home/hailo/giroscopio_compiled.har
```

! El modelo usa 2 contextos (normal)

El Hailo-8 tiene que partir el modelo en dos contextos (context_0 y context_1) porque no cabe en uno solo. Esto es normal para YOLOv8-Pose. La utilización total es ~60% control, ~45% compute, ~40% memoria.

4.7 Guardar archivos y salir del contenedor

Con `--rm` los archivos de `~` se pierden al hacer exit. Antes de salir, copia todo a `/workspace` (que es tu carpeta `hailo_workspace` en Windows):

```
# Verificar que los archivos existen y tienen tamaño razonable
ls -lh ~/giroscopio.hef ~/giroscopio_optimized.har ~/giroscopio_compiled.har

# Copiar a /workspace (visible en Windows como hailo_workspace\)
cp ~/giroscopio.hef /workspace/
cp ~/giroscopio_optimized.har /workspace/
cp ~/giroscopio_compiled.har /workspace/
```

```
cp ~/giroscopio.alls /workspace/

# Verificar que llegaron
ls -lh /workspace/*.hef /workspace/*.har /workspace/*.alls

# Solo cuando veas todos los archivos en /workspace, salir
exit
```

Tamaños esperados:

-rw-rw-rw-	1	hailo	ht	7.1M	giroscopio.hef	← ARCHIVO FINAL
-rw-rw-rw-	1	hailo	ht	62M	giroscopio_optimized.har	← modelo cuantizado
-rw-rw-rw-	1	hailo	ht	69M	giroscopio_compiled.har	← modelo compilado
-rw-rw-rw-	1	hailo	ht	512B	giroscopio.alls	← script optimización

⚠ No hagas exit hasta ver los archivos en /workspace

El comando exit con --rm es irreversible. Una vez ejecutado, todo lo que esté en ~ desaparece para siempre. Verifica SIEMPRE con:

```
ls /workspace/*.hef
```

antes de salir. El .hef debe aparecer en C:\Users\TuUsuario\hailo_workspace\ en Windows.

PARTE 5 — Copiar el .hef a la Raspberry Pi 5

Una vez fuera del contenedor, el archivo giroscopio.hef está en `C:\Users\TuUsuario\hailo_workspace\`. Cópialo a la Raspberry Pi 5 usando uno de estos métodos:

Método A — SSH/SCP desde PowerShell (Windows 10/11)

Windows 10 y 11 incluyen OpenSSH por defecto. Desde PowerShell:

```
# Sustituye <IP-RASPBERRY> por la IP real de tu RPi5
# Ejemplo: 192.168.1.45
scp C:\Users\TuUsuario\hailo_workspace\giroscopio.hef pi@<IP-RASPBERRY>:~/

# Para encontrar la IP de la Raspberry Pi:
# En la RPi: hostname -I
# O mira en tu router la lista de dispositivos conectados
```

¡ SSH debe estar habilitado en la Raspberry Pi

En la RPi5, habilita SSH desde: `sudo raspi-config` → Interface Options → SSH → Enable

O durante la instalación de Raspberry Pi OS desde el Raspberry Pi Imager:

Services → Enable SSH → Use password authentication

Método B — WinSCP (interfaz gráfica)

WinSCP es una aplicación gratuita con interfaz de arrastrar y soltar. Ideal si no te sientes cómodo con la línea de comandos.

- Descarga WinSCP desde: <https://winscp.net/>
- Instala y abre WinSCP
- Nueva sesión: Protocol=SFTP, Host=<IP-RASPBERRY>, User=pi, Password=<tu_password>
- Conéctate y navega a la carpeta home (~) de la RPi
- Arrastra giroscopio.hef desde Windows a la carpeta de la RPi

Método C — Pendrive USB

Si no tienes conexión de red entre el PC y la Raspberry Pi:

- Copia giroscopio.hef a un pendrive USB
- Conecta el pendrive a la Raspberry Pi 5
- En la RPi: `cp /media/pi/<pendrive>/giroscopio.hef ~/`

PARTE 6 — Referencia rápida

6.1 Chuleta: secuencia completa en PowerShell

Copia y pega esta secuencia completa en PowerShell. El único paso interactivo es el parser (puede preguntar si reintentar — responder y). NO incluye exit al final para evitar salir antes de verificar.

PASO PREVIO (fuera del contenedor) — Arrancar Docker:

```
# En PowerShell - sustituir TuUsuario
docker run -it --rm `
  -v C:/Users/TuUsuario/hailo_workspace:/workspace `
  hailo8_ai_sw_suite_2025-10:1 bash
```

DENTRO DEL CONTENEDOR — Pipeline completo:

```
cd ~

# PASO 1: Convertir imágenes de calibración a .npz
python3 << 'PYEOF'
import os, numpy as np
from PIL import Image
calib_dir='/workspace/calib/'; out_dir=os.path.expanduser('~/.calib_npy/')
os.makedirs(out_dir, exist_ok=True)
files=[f for f in os.listdir(calib_dir) if f.lower().endswith(('.jpg','.png'))]
for i,f in enumerate(files):
    img=Image.open(os.path.join(calib_dir,f)).convert('RGB').resize((640,640))

    np.save(os.path.join(out_dir,f'calib_{i:04d}.npz'),np.array(img,dtype=np.float32))
print(f'OK: {len(files)} archivos .npz')
PYEOF

# PASO 2: Parsear .onnx → .har
hailo parser onnx /workspace/giroscopio_pose_best_n_g1.onnx \
  --net-name giroscopio \
  --end-node-names \
    /model.22/cv2.0/cv2.0.2/Conv \
    /model.22/cv3.0/cv3.0.2/Conv \
    /model.22/cv4.0/cv4.0.2/Conv \
    /model.22/cv2.1/cv2.1.2/Conv \
    /model.22/cv3.1/cv3.1.2/Conv \
    /model.22/cv4.1/cv4.1.2/Conv \
    /model.22/cv2.2/cv2.2.2/Conv \
    /model.22/cv3.2/cv3.2.2/Conv \
```

```
/model.22/cv4.2/cv4.2.2/Conv
```

```
# PASO 3: Crear script .alls
```

```
cat > ~/giroscopio.alls << 'EOF'
```

```
normalization1 = normalization([0.0, 0.0, 0.0], [255.0, 255.0, 255.0])
```

```
change_output_activation(conv71, sigmoid)
```

```
change_output_activation(conv58, sigmoid)
```

```
change_output_activation(conv44, sigmoid)
```

```
pre_quantization_optimization(equalization, policy=disabled)
```

```
quantization_param(output_layer3, precision_mode=a16_w16)
```

```
quantization_param(output_layer6, precision_mode=a16_w16)
```

```
quantization_param(output_layer9, precision_mode=a16_w16)
```

```
post_quantization_optimization(finetune, policy=enabled, learning_rate=0.00015)
```

```
EOF
```

```
# PASO 4: Optimizar (~13 min)
```

```
hailo optimize ~/giroscopio.har \
```

```
--hw-arch hailo8 \
```

```
--calib-set-path ~/calib_npy/ \
```

```
--model-script ~/giroscopio.alls
```

```
# PASO 5: Compilar (~8 min)
```

```
hailo compiler ~/giroscopio_optimized.har --hw-arch hailo8
```

```
# PASO 6: Verificar y copiar a /workspace
```

```
ls -lh ~/giroscopio.hef ~/giroscopio_optimized.har ~/giroscopio_compiled.har
```

```
cp ~/giroscopio.hef /workspace/
```

```
cp ~/giroscopio_optimized.har /workspace/
```

```
cp ~/giroscopio_compiled.har /workspace/
```

```
cp ~/giroscopio.alls /workspace/
```

```
ls -lh /workspace/*.hef /workspace/*.har
```

```
# — SOLO CUANDO VEAS LOS ARCHIVOS EN /workspace —————
```

```
exit
```

```
# — DE VUELTA EN POWERSHELL: copiar a Raspberry Pi 5 —————
```

```
scp C:\Users\TuUsuario\hailo_workspace\giroscopio.hef pi@<IP-RPi>:~/
```


6.2 Tabla de errores y soluciones

Error	Causa	Solución
pull access denied	Tag :latest no existe en local	Usar hailo8_ai_sw_suite_2025-10:1 (con tag :1)
Cannot connect to the Docker daemon	Docker Desktop no está abierto	Abre Docker Desktop y espera a que arranque
Permission denied pyhailort.log	/workspace es read-only	Ejecutar SIEMPRE desde ~, nunca desde /workspace
StopIteration npy_dir_to_dataset	calib/ contenía JPGs, no .npz	Convertir con el script Python antes del optimize
Couldn't detect CalibrationDataType	Carpeta .npz vacía o mal generada	Verificar con ls ~/calib_npz/ que hay archivos .npz
--calib-path not recognized	Parámetro incorrecto	Usar --calib-set-path (no --calib-path)
Model requires quantized weights	Se compiló el .har sin optimizar	Ejecutar hailo optimize ANTES de hailo compiler
No valid partition found	Nodos finales del parser incorrectos	Usar los 9 nodos cv2/cv3/cv4 Conv con --end-node-names
FileNotFoundError giroscopio.har	Parser ejecutado desde directorio incorrecto	Hacer cd ~ antes de ejecutar el parser
exit prematuro, archivos perdidos	--rm elimina todo al salir	Verificar ls /workspace/*.hef antes de hacer exit
docker: command not found	Docker Desktop no instalado o no en PATH	Reinicia PowerShell tras instalar Docker Desktop
invalid volume specification	Ruta Windows mal formateada	Usar barras / en la ruta: C:/Users/... no C:\\Users\\...

6.3 Archivos generados y para qué sirve cada uno

Archivo	Tamaño	Descripción
giroscopio.har	~13 MB	Modelo parseado (FP32). Salida del parser.
giroscopio.alls	~512 B	Script de optimización. Guardar para reutilizar.
calib_npy/	variable	Imágenes en .npy. Se regenera en cada sesión.
giroscopio_optimized.har	~62 MB	Modelo cuantizado INT8 con fine-tuning. Guardar.
giroscopio_compiled.har	~69 MB	Modelo compilado con mapeo al chip. Para depuración.
giroscopio.hef	7.1 MB	ARCHIVO FINAL. El único que se carga en la Raspberry Pi 5.

— fin del tutorial —